

— 开源之夏 2026 · 项目申请书

为 Easy-Es 引入一套 AI Harness 自动化 PR 流水线

从社区噪声到可合并代码：一条 human-in-the-loop 的半自动演进链路

牛晨勋 · 中南大学

2026.09

开源组织: Dromara | 导师: 老汉

难度: 进阶 | 技术领域: Database

| 目录

01	项目基本信息	3
02	项目理解	4
03	前期调研	6
04	业界先例与设计依据	8
05	个人开源经历与项目匹配度	9
06	实现难度评估	11
07	基础实现目标	13
08	技术方案	13
09	实施计划	17
10	四个月进度安排	18
11	预期产出	20
12	验收标准	21
13	范围边界与可行性结论	22

项目基本信息

申请项目	为 Easy-Es 引入一套 AI Harness 自动化 PR 流水线
项目难度	进阶(经典模式, 结项劳务报酬税前 5000 元)
技术领域	Database 编程语言: Java / Python 开源协议: Apache-2.0
开源组织	Dromara
项目导师	老汉(1035676261@qq.com)
成果仓库	https://gitee.com/dromara/easy-es
开发周期	4 个月(以项目详情页标注为准, 自中选之日起算)
支持架构	RISC-V64
申请人	牛晨勋(GitHub: Stelquis CNB: cnb.cool/u/Stelquis)
就读院校	中南大学 · 数据科学与大数据技术 · 本科(2023.09 至 2027.06)
投入估算	约 17 周, 按每周 30 小时计, 合计约 510 小时

项目背景

任务书对项目背景的表述为: Easy-Es 是一款基于 Elasticsearch 的国产 ORM 框架, 致力于降低开发者使用 ES 的门槛; 传统的社区维护模式依赖核心作者人工处理 Issue、设计功能、编写代码, 面临响应速度慢、人力成本高的问题; 利用大模型(LLM)与自动化工具(Harness)辅助甚至主导项目的演进, 实现「AI 驱动的开源项目自治」, 已成为提升社区活力的重要趋势。

任务书对已有工作的说明为: 社区已积累一定的用户群体与 Issue 反馈, 作者目前以手动方式在 GitHub/Gitee 上收集需求, 并在本地进行架构设计与代码提交; 业界已有部分项目开始尝试 AI 辅助编程(如 GitHub Copilot), 但在「全自动化的需求挖掘到 PR 提交」这一完整链路(Harness)上尚处于探索阶段。

项目目标

任务书「最终项目实现的目标」列出三条, 本项目对每一条的落点如下:

任务书目标	本项目落点
自动化闭环: 打通「信息爬取 → 需求分析 → 架构设计 → 代码生成 → 自动 PR」的完整链路	M1 至 M6 六个模块首尾相接, M7 横切管控, 全链路由统一状态机编排(\$08)
社区减负: 使作者从繁琐日常维护中淡出, 转为通过配置与审核 AI 的产出	三处强制人工门禁 + 每步产出落盘为结构化产物, 审核对象是改动清单与验证报告, 而非逐行读代码(\$08)
可落地的 Demo: 至少实现 1 至 2 个由 AI 自动挖掘并生成的、高质量的功能模块 PR	W16 产出 2 个 PR, 需求可溯源至具体 Issue 编号(\$10、\$11)。2 个为达标口径, 保底承诺 1 个, 见 \$07

五项产出要求与实施要点

任务书列出五项产出要求, 其实施要点如下:

#	产出要求	优先级	实施要点
1	基于 Skills 的爬虫，抓 GitHub/Gitee Issue 及 ES 官方动态	高	技术成熟，难点在数据质量而非抓取本身
2	AI 自动选题与进化方向挖掘，支持人工审核与自主指定	高	全链路中最依赖领域判断的一环
3	架构与详细设计文档自动生成 + 自动 Review + 人工把关	中	Java 隐式约定多，生成易、正确难
4	代码实现 + 单测 + 自动 PR 端到端闭环	高	主战场，工作量占比最高
5	使用说明文档 + 基于真实需求演示生成至少 1 个高质量 PR	高	项目成败判据

产出 5 的「基于真实需求」排除了构造演示需求走通流水线的做法，对应验收要求是：每个 AI 生成的 PR，需求可溯源到具体 Issue 编号或 ES 官方动态条目。

产出 2 的「进化方向挖掘」若由 LLM 自由发挥，会产出「建议增强可观测性」这类 无法证伪也无法落地的建议。该环节产出将强制结构化，不是自由文本，而是带证据链的候选列表。

相关仓库

仓库	地址	用途
Easy-Es 主仓(成果仓库)	https://gitee.com/dromara/easy-es	PR 提交目标；能力图谱语料来源；默认分支 master
Easy-Es GitHub 镜像	https://github.com/dromara/easy-es	Issue 第二数据源；默认分支 main
Harness 系统(拟新建)	gitee.com/dromara/easy-es-harness	本项目系统代码，归属方案见 §09，待确认事项见 §10 Q4

02 · PROBLEM DEFINITION

项目理解

任务书所述三条不足的核实

任务书「存在的不足」列出三条。逐条以公开数据核实如下：

任务书所述不足	核实结果	来源
需求响应滞后 依赖作者人工浏览 Issue 与跟踪 ES 官方动态，效率低下、易遗漏高价值需求	Gitee 214 + GitHub 104 个 open issue，近期维护者回复率接近 0；其中 #143 / #135 / #157 三条需求实际已实现，至今仍为 open(\$03)	Issues
维护压力集中 核心作者承担需求分析、架构设计、编码测试的全部工作	GitHub contributors 仅 1 人(xpc1024 ，77 commits, 占比 100%)；提交邮箱由 xpc@dromara.org 转为 xingpc37977@hundsun.com（恒生电子，全职）	GitHub API / commit 记录
缺乏自动化流程 没有系统能自动抓取互联网信息、结合现有框架功能智能选题，并自动完成设计文档与代码实现	近 4 个月无实质代码提交(期间仅 2026-08-28 一次 README 改动)； feature-boot4 分支已创建但未合入，Spring Boot 4 适配停滞；任务书亦指出该完整链路尚处于探索阶段	Gitee / GitHub

另有一组数据佐证任务书所述「迭代速度可能放缓」：v3.0.2（2026-05-10）仅含 1 条 bugfix，为项目史上最小 release。

三条判断

判断一 · 项目属于元工程，验收是非线性的

交付物不是单个功能，而是能产出功能的系统。即便 90% 的模块正常运行，只要产出的 PR 是垃圾，整体价值就归零。因此验收以产出质量而非功能完成度为准。

判断二 · 脏活占比约 80%

建验证闭环 25% | 搭环境 25% | 数据清洗与标注 15% | 能力图谱 15% | 调 LLM / Prompt / RAG 15% | 架构与设计创新 5%。

搭环境与建验证闭环合计占 50%，加上数据清洗与标注后达 65%；再计入能力图谱，即前两个月所覆盖的 80%。这一分布决定了排期：前两个月先做环境与验证，而非先搭 Agent 框架。

判断三 · 难度集中在三处

- ① LLM 能否生成通过 Easy-Es 全套测试的 Java 代码，取决于验证反馈设计
- ② 能否筛出「值得做且 AI 能做对」的需求，取决于领域判断
- ③ RISC-V64 的验收口径，需与导师确认(\$10)

前期调研

数据获取时间：2026 年 9 月 4 日。

调研范围由任务书的产出要求倒推确定：要自动生成代码(产出要求 4)，须先掌握 Easy-Es 的改动链路与领域约束，否则生成结果无法通过社区测试；要自动选题(产出要求 2)，须先摸清 Issue 候选池的实际成分，否则评分器无从设计；要抓取 Issue 与 ES 官方动态(产出要求 1)，须先确认公开语料中哪些已过时或自相矛盾。以下四项实测分别对应上述三个问题。

代码库体量与改动链路

体量(v3.0.2 标签实测)：Java 主源码 198 个文件、**24,647 行**；测试 49 个文件、7,254 行。 easy-es-core 独占 17,696 行(71.8%)，是能力图谱与代码生成的主体。条件构造器对外暴露 **898 个方法** (Compare 321 / Func 304 / Geo 150, 其余 123)； FieldType 枚举 27 个值，注解 14 个。

改动链路：新增一个查询 API 需横跨 5 个文件层级。

层	文件	职责	行数
①	conditions/function/Func.java (或 Compare.java)	接口声明 + javadoc	1790 / 2215
②	kernel/AbstractWrapper.java	实现 + 参数校验	1027
③	kernel/AbstractChainWrapper.java	Lambda / 链式重载	791
④	kernel/WrapperProcessor.java	ES DSL 序列化	1101
⑤	toolkit/FieldUtils.java	.keyword 后缀判定	292

漏任何一处都会导致能编译但语义错误：编译器不会报错，只有测试能发现，而测试失败信息未必能反推出根因。

LambdaEsQueryWrapper 全文仅 53 行、4 个方法 (2 个公有构造器、1 个包内构造器、1 个 instance() 覆写)，不含任何查询方法；其父类 AbstractLambdaQueryWrapper 是 13 行空壳，实现在 AbstractWrapper 。按「在 LambdaEsQueryWrapper 里加方法」的直觉去改，会直接改错文件。

改动高度集中：Compare + Func + AbstractWrapper + WrapperProcessor 共 6,133 行(占主源码 25%)，是几乎每个 API 改动都要触碰的窄口，L1 定向单测优先覆盖这四个文件。

其余领域约束

- `.keyword` 智能后缀: 按字段索引类型 + 查询类型上下文判定, 逻辑在 `FieldUtils.java:210-237` , 可用 `smartAddKeywordSuffix=false` 关闭。v3.0.0 release note 中「修复聚合 / 排序 / `prefix` 查询时部分字段未加 `.keyword` 后缀」佐证其易错性。
- `id` 列约定: v3.0.0 起支持自定义名称, 不再强制 `id` , 但默认约定仍为 `id` 。2.x 文档中「实体类必须含 `String id` 」已过时。
- 依赖版本: 底层为 `elasticsearch-java` , 锁死 **7.17.28**, 涉及 `elasticsearch-java` / `elasticsearch` / `elasticsearch-geo` 三个构件; Spring Boot 内置 `starter` 需排除后显式声明。ES 服务端 7.x / 8.x 均可, JDK 兼容 8 / 17 / 22。
- 覆盖率基线: 官方宣称单元测试综合覆盖率超 **95%**。这既提供了 L1 定向单测的高质量语料库, 也意味着新增 PR 不附带测试即不符合社区标准。

版本节奏: v3.0.0(2025-05-11)为架构级重构(换 `elasticsearch-java 7.17.28`、弃 `fastjson` 换 `jackson`), 2.x 资料已过时; v3.0.1(2026-02-27)引入向量检索 `beta`; v3.0.2(2026-05-10)仅 1 条 `fix`。

Issue 生态实测

对 GitHub 最新 30 条 open issue(#135 至 #171, 2024-09 至 2026-08)人工分类:

类型	数量	占比	示例
feature request	8	约 27%	#169 suggest 自动补全、#168 nginx pathPrefix、#156 flattened 类型、#154 <code>@IndexField(index=true)</code> 、#147 动态数据源、#146 时区与缺失的 <code>ge</code> 方法
bug	9	约 30%	#167 管道聚合从第 3 个字段起丢失、#164 <code>_score</code> 排序被强制排到末尾、#170 <code>map</code> 字段序列化
求助 / 噪音	13	约 43%	#171 外部安全问卷、#166「支持 SpringBoot4 吗」、#159 https 如何配置、#141 IDEA 报红

实测分布与依据《避坑指南》作者自述所作的估计(可转化需求低于 10%)差异较大。据此确定两点: 候选池充足(feature 加 bug 合计约 57%), 候选需求不足的风险下调至低; 噪声仍占 43%, 分类器保留。相应地, W4 先人工标注 200 至 300 条样本再训练分类器, 而非依赖先验推测。

三处待澄清的不一致

一 · 底层客户端表述在 5 处公开材料中各不相同

Gitee README 正文已更新为 ElasticsearchClient，而仓库简介、GitHub README、Gitee 项目标签、官网「特性」区仍为 RestHighLevelClient。

若爬虫语料混入仍在流通的 2.x 文档，检索结果会自相矛盾，故能力图谱以 3.x 源码为唯一事实来源。

二 · ISSUE 状态与代码实际能力脱节

#143「需要 es8.0 以上支持」已由 v3.0.0 完成；#135「能否支持 es8.x 的向量搜索」已由 v3.0.1 引入 beta；#157「createIndex() 无法重复使用」已由 v3.0.1 修复。三者至今仍为 open。

这说明「某功能是否已实现」即使对 Maintainer 也不易判定，是能力图谱闭环的直接依据。

三 · RISC-V64 验收口径

任务书标注「支持架构：RISC-V64」，官方 README 宣称「兼容 jdk8 至 jdk17 支持 RISC-V 架构」（同一行又写「兼容 jdk8~jdk22」），而 Elastic 官方支持矩阵无 riscv64 构建。处理方案见 §10。

04 · PRIOR ART

业界先例与设计依据

设计依据：任务书对质量把关的规定

任务书「希望改进的点」在质量把关一条中写明：「每个核心阶段（设计、编码、测试）的产出均需经过『自动 Review + 人工审核』双重把关，确保代码质量。」

同节对端到端交付的描述为：实现从「需求」到「代码合并」的半 / 全自动化流转。

据此，人工审核是任务书规定的交付形态之一，不是为降低实现难度所作的让步。两类把关的分工是：自动 Review 承担可机械判定的部分（编译、测试、覆盖率、静态扫描、设计声明与实际 diff 的一致性），人工审核 承担需要领域判断的部分（选题价值、语义正确性、是否可合并）。二者串行叠加，不是二选一。

任务书同时界定了业界现状：「已有部分项目开始尝试使用 AI 辅助编程（如 GitHub Copilot），但在『全自动化的需求挖掘到 PR 提交』这一完整链路（Harness）上尚处于探索阶段。」本项目的落点即在这条完整链路上交付一套 可运行系统，而非再做一层代码补全工具。

业界已有的分段先例

先例	覆盖的段	可借鉴处
SWE-agent	Issue → 补丁 → 测试	分层验证的环境抽象
OpenHands	Issue → PR 全流程	端到端编排的状态管理
AutoCodeRover	Issue → 定位 → 补丁	Java 代码结构化检索
AutoPR	Issue → 修复 → PR	自动 PR 描述规范
Issue2Idea	Issue → 需求挖掘	需求挖掘侧最接近的先例
PR-Agent / PR Pilot	PR → 自动评审	自动 Review 环节

无人值守的风险与防控

任务书「质量把关」条要求确保代码质量。自动化链路与人工作流程的关键差异在于：一旦无人值守，模型的行为漂移、递归循环与 对外写入动作都可能在无人察觉的情况下持续累积，其共同点是系统自身无法判断已经偏离目标。

因此本项目的防控不以「模型足够可靠」为前提，而以可降级、可熔断(P4) 为设计约束：重试次数设上限，重复输出相似度检测即熔断，每步产出落盘可回溯，对外的写入动作(分支、Commit、PR、评论)默认不自动执行。对应实现见 §08 与 §09。

05 · FIT

个人开源经历与项目匹配度

技术要求对照

任务书「项目技术要求」共三条，逐条对照如下(首列为该条在任务书中的编号)：

技术要求	任务书要求	对应经历
1	了解 Python 编程	Python 为主要开发语言(梨园星图 30+ 脚本管线、PySpider 16 万页爬虫)
1	了解 Java 基础	Java 为已修课程与列明技能; W1 起进入 Java 工程上下文, W2 手工 PR 补齐
1	熟悉 AI 工程化语言特性	梨园星图集成 BGE 嵌入 + KeyBERT + SimCSE, 用 DeepSeek + FastAPI + LangChain 做别名消歧与关系标注; DeepAstraDraft 构建 123 项参数语义索引
1	熟练掌握 vibe coding	见下方
2	熟悉 LLM 应用开发、Prompt 工程与 RAG	同上; DeepAstraDraft 设计「意图分类 → 规则匹配 → LLM 增强」渐进式推理, 58 题测试集准确率 93.1%
2	了解爬虫技术	PySpider: 生产者-消费者架构, 44 线程, 64 分片去重器(锁竞争降低 98%, 查询小于 1ms, 10 万 URL / 800MB)
3	有 AI 辅助编程或自动化流水线经验更优	DeepAstraDraft: LangChain + LangGraph 实现 4 子 Agent 协作管线 + 5 个领域 Skill, 含无 API Key 时的纯规则降级路径

vibe coding 实践

该要求的关键在于在 AI 主导代码产出的模式下保证交付质量。相关实践：

- DeepAstraDraft(CAD 图纸智能问答 Agent)：开发过程用自然语言描述 各 Agent 职责与协作方式、由模型产出实现、依据 58 题测试集结果迭代，最终准确率 93.1%。
- Neural-Link OS Agent：智能系统管理代理，自然语言驱动命令生成。该项目界定了 vibe coding 的适用边界：当产出物会改变系统状态时，逐行审查是必要的，因此设计了四级风险控制。

该边界判断在项目中的技术实现见 §08。

相关经历

DEEPASTRADRAFT: 多 AGENT 编排与降级设计(最直接相关)

两点与本项目对应：多阶段管线的状态管理；无 API Key 时降级至纯规则模式（保持同等准确率），后者是 P4 可降级可熔断原则的实践原型。

湖南电信实习：分类器调试

从 48 万条生产工单中清洗提取 2.7 万条训练数据，用 StructBERT + LoRA 做五类意图分类。关键产出是定位并修复「词表文件损坏导致中文全编码为 [UNK]」这一根因，使 Macro-F1 从 0.14 升至 0.81，三种子集成达 0.8909。

对 M1 分类器的价值在于：分类器效果不佳时，问题常在数据处理链路而非模型。另一项工作（定位 GCJ02 / WGS84 坐标系混用导致的 676m 系统偏移）同样针对「能跑通但结果是错的」这类隐蔽缺陷，与 §06 N1 同形态。

梨园星图：大规模非结构化数据处理(CHINAVIS 2026 二等奖)

针对 1,473 部京剧剧本搭建 Python 管线(30+ 脚本)，提取 29 维角色特征、12 维主题、8 项网络拓扑指标，构建 7,965 角色人次的共现网络，并用卡方检验、ANOVA、KMeans、PCA + 层次聚类完成模式发现。

与本项目 M1 数据清洗、M2 能力图谱构建属于同类工作：非结构化数据的结构化治理。

短板与补强

短板	补强计划
ES 领域深度不足 (text/keyword 语义、DSL 树、mapping 迁移)	W1 至 W2 精读《避坑指南》与文档，配合手工 PR 实战；这是 N1 的根本缓解
Java 工程经验浅于 Python	W1 起进入 Easy-Es 工程上下文；Maven 多模块与依赖冲突排查通过实战补齐，卡点超 3 天求助
首次参与 Dromara 社区	W2 的破冰 PR 为融入动作；全程保持周度沟通

06 · DIFFICULTY

实现难度评估

已有基础

调研已完成，W1 无需摸底。申请前已对 Easy-Es v3.0.2 完成实测：体量统计、5 层改动链路追踪、30 条 Issue 人工分类、5 处文档一致性核对、发布节奏梳理(§03)。W1 可直接进入环境搭建。

四条可直接复用的工程经验(各项实测指标见 §05)：

已有经验	对应本项目环节
PySpider 分布式爬虫	M1 感知层 (W3)
湖南电信工单分类器	M1 分类器 (W4)
梨园星图数据管线	M1 数据清洗、M2 能力图谱 (W5)
DeepAstraDraft 多 Agent 编排	M3/M4 编排与 P4 降级 (W10 至 W14)

环境就绪。Docker 29.6.2 实测可拉取并运行容器，Testcontainers 起真实 ES 已具备条件 (L2 依赖此项)；Python 3.12 已装；Gitee / GitHub / DockerHub 网络可达。缺口仅 Java 与 Maven，W1 补齐。

PR 候选已就绪。已识别 8 个 feature request (#169 / #168 / #156 / #154 / #147 / #146 等) 与 9 个 bug (#167 / #164 等)。W16 的选题不需要从零开始找需求。

主要难点

#	难点	为何难	应对
N1	Java 语义错误隐蔽	编译期与类型系统均不提供信号 (§03)	分层验证 + 客观信号门禁；测试强制覆盖新增路径
N2	5 层链路一致性	漏改不报错，且实现不在 LambdaEsQueryWrapper (§03)	AST 对称性检查；改动清单与 diff 比对
N3	验证速度 vs 迭代次数	端到端预估需 6 至 35 次迭代，而全量测试耗时 5 至 15 分钟 (§08)	L0/L1/L2 分层，把错误发现成本压到秒级 / 分钟级
N4	噪声筛选	求助 / 噪音占 43%，直接喂给 LLM 会放大误判	规则前置 + LLM 兜底四分类；人工标注 200 至 300 条样本
N5	ES 领域知识门槛	同 §05「短板与补强」第一条	见 §05 补强计划 (W1 至 W2 集中补，W2 手工 PR 验证)
N6	PR 合并窗口	若跨春节，Maintainer 审核停滞，而结项以合并时间为准	W16 前置到 W14 末启动，预留 3 周合并窗口

文档中三处出现的迭代次数口径不同：N3 的 6 至 35 次为端到端全流程轮次的 预估；§08 架构图中的 15 次为 M5 验证失败后反馈至 M4 的重试上限，由 M7 管控层设定、状态机执行；§12 的 8 至 15 次为分层验证达成设计指标后的目标均值。

基础实现目标

任务书「最终项目实现的目标」第三条要求：在开源之夏期间产出一套可落地的 自动化流程，至少实现 1 至 2 个由 AI 自动挖掘并生成的高质量功能模块 PR。本项目承诺的最小可交付集合(对应 §12 的「保底」档)如下：

- 1. Harness 全链路可运行：M1 至 M7 均可执行，端到端演示通过，对应任务书「自动化闭环」一条所述完整链路
- 2. 1 个手工 PR 被合并：用于对齐导师偏好与社区规范，并为 AI 生成的 PR 打通合并通道
- 3. 至少 1 个 AI 生成的 PR 通过全量测试并提交，需求可溯源到具体 Issue 编号，对应任务书「至少 1 至 2 个」口径的下限
- 4. 使用说明文档齐备：含一键启动(Docker Compose)与演示脚本，对应产出要求 5

任务书口径的上限(2 个 PR, 且为 AI 自动挖掘并生成)不列为保底，而列为 §12 的达标档：保底只承诺下限，达标承诺 2 个 PR 且至少 1 个被合并。

技术方案

设计原则

#	原则	含义
P1	客观信号优先	质量判定优先采信编译、测试、覆盖率、静态扫描等硬信号；LLM 评审不作放行依据
P2	human-in-the-loop 而非 human-on-the-loop	人工介入是流程节点，不是异常时的补救。三处门禁覆盖任务书「质量把关」条要求的设计、编码、测试三阶段：设计确认对应设计阶段；PR 提交前确认一次审核编码与测试两阶段的产出(diff 与测试报告)。两者由 M4、M5 连续产出且紧邻，合并在同一节点审核，避免为形式上凑齐三处而将人工介入碎片化
P3	反馈密度优先于编排复杂度	依据 §03 实测(漏改不报错、仅测试可发现)，工程投入放在「让模型更快知道自己错了」
P4	可降级、可熔断	任何环节失败都能安全停在当前状态
P5	不追求通用	只针对 Easy-Es 单仓库做深

系统架构

任务书「希望改进的点」列出五项，本项目的模块划分即由这五项逐一推出，不另设与任务书无关的能力：

任务书「希望改进的点」	对应模块
智能感知：通过 Skills 自动爬取 GitHub/Gitee 仓库 Issue，并监控 Elasticsearch 官方发展动向	M1 感知层 (U1 Issue 抓取器、U2 四分类器)+ issue-crawler 技能单元
自动选题与决策：结合当前框架已实现的功能，AI 自动评估并优先 筛选高价值需求；支持触发人(Maintainer)从推荐列表中选择，或由用户自主指定功能选题	M2 选题层 (U3 能力图谱构建器、U4 选题评分器)；「从推荐列表选择 / 自主指定」由人工门禁 1 选题确认承担
AI 深度挖掘与演进：对选定需求深度挖掘、产出进化方向，并依次自动完成架构设计、详细设计	M3 设计层 (U5 设计生成器、U6 一致性校验器)
质量把关：每个核心阶段(设计、编码、测试)的产出均需经过「自动 Review + 人工审核」双重把关	M5 验证层 (U9 / U10 / U11 承担自动 Review)+ 三处人工门禁承担人工审核
端到端交付：自动完成代码实现、单元测试，并生成 Pull Request，实现从「需求」到「代码合并」的半 / 全自动化流转	M4 生成层 (U7 代码生成器、U8 单测生成器)+ M6 交付层 (U12 PR 组装器)

唯一不对应任务书任一条的是 M7 管控层。它不提供功能，而是任务书「质量把关」条中「确保代码质量」得以成立的前提：自动化链路在无人值守时无法自行判断是否已偏离目标(\$04)，故熔断、重试上限与全链路审计属安全约束，不是可选增强。

M7 管控层 Guardrails					
Token 上限 · 执行超时 · 重复输出熔断 · 死循环检测 · 全链路审计 (横切约束 M1 至 M6)					
M1 感知层 Gitee Issues GitHub Issues ES Release 动态 规则 + LLM 分类	M2 选题层 能力图谱 RAG 是否已实现校验 可测性 / 价值评分 候选排序	M3 设计层 架构设计生成 详细设计生成 一致性校验 AST 对称性检查	M4 生成层 Java 代码生成 单元测试生成 基于 diff 定向 领域硬约束注入	M5 验证层 L0 编译 < 30s L1 定向 < 2min L2 全量回归 客观信号采集	M6 交付层 分支管理 Commit 规范 PR 描述生成 状态追踪
统一状态机编排(Python) · 每步产出落盘为结构化 JSON / Markdown					
显式状态迁移 + 重试上限 + 失败分支，非 Agent 自主决策；M5 验证失败时沿虚线反馈至 M4 重试(上限 15 次)					
人工门禁 1 选题确认		人工门禁 2 设计确认		人工门禁 3 PR 提交前确认	

图 1 系统分层架构。M5 为技术内核：M1 至 M4、M6 均可降级或以人工接管兜底，而 M5 的反馈密度决定整条链路能否在重试上限内收敛。

编排采用显式状态机而非 Agent 自主决策，取的是 可预测、可回放、易熔断：失败可定位到具体步骤并重放至出错点。三处人工门禁为强制流程节点，对应 P2(选型对比见「方案选型」)。

实现单元分解

#	实现单元	输入	产出	通过标准
U1	Issue 抓取器	Gitee/GitHub API、ES Release Notes	SQLite 候选池	增量同步无遗漏，字段完整
U2	四分类器	候选池原始条目	feature_request / bug / question / noise	准确率 ≥ 85%
U3	能力图谱构建器	3.x 源码(core / annotation / common)	两级切分索引 + BGE/BM25 混合检索	30 至 50 条测试集命中率 ≥ 80%
U4	选题评分器	候选 + 检索结果	八字段结构化条目(含 source_issue)	需求 100% 可溯源
U5	设计生成器	选定需求 + 相关代码上下文	architecture.md / detailed_design.md	改动清单完整
U6	一致性校验器	设计声明 vs 实际 diff	校验报告	能检出漏改的链路层级
U7	代码生成器	设计文档 + 硬约束	基于 diff 的定向改动	改动面不超出设计声明
U8	单测生成器	新增代码	对应测试用例	覆盖率不低于现有 95% 基线
U9	L0 验证器	编译产物	编译 + checkstyle/spotbugs + 依赖校验	耗时 < 30s，拦截率 ≥ 60%
U10	L1 验证器	受影响模块	定向单测结果	耗时 < 2min，再拦截 ≥ 25%
U11	L2 验证器	全仓	Testcontainers 起真实 ES 的回归结果	覆盖三条测试线
U12	PR 组装器	通过验证的 diff	分支 / Commit / PR 描述(关联 Issue)	符合社区 PR 模板
U13	管控器	全流程	Token / 超时 / 熔断 / 审计	重复输出相似度检测先于重试上限(15 次)触发熔断

方案选型

方案	优点	缺点	选择
A: Agent 自主决策	灵活，可处理未预期情况	行为不可预测，易失控	否
B: 显式状态机	可预测、可回放、易熔断、易调试	灵活性受限，需预定义路径	选 B
C: 整文件重写	实现简单	易引入无关变更，diff 噪声大	否
D: 基于 diff 定向修改	改动面可控、diff 可审计	需准确定位插入点	选 D
E: 纯向量检索	语义匹配好	对 Java 标识符等强关键词召回率低	否
F: BM25 + 向量混合	符号与语义兼顾	实现稍复杂	选 F

选 D 并用 AST 定位辅助插入点确定(借鉴 AutoCodeRover 思路)。

语言与工具选型

Easy-Es 是 100% Java 仓库，但任务书「编程语言」一栏同时列出 JavaPython，且产出要求第一条为「开发基于 Skills 的爬虫模块」。本项目编排层用 Python、产出代码用 Java，两层各司其职。

三条佐证：任务书「编程语言」同列 JavaPython；措辞差异明显（「了解 Python 编程」对「了解 Java 基础」）；技术要求要求「熟悉 AI 工程化语言特性」，该生态事实标准语言为 Python。

Java 侧难度不因主体用 Python 而降低。5 层改动链路与 .keyword 约定 全部落在 Java 侧，是主要技术风险(N1 / N2)，W1 至 W2 集中补强。

工具链：Python 3.12 + 显式状态机；可插拔 LLM 适配层（默认 DeepSeek / Qwen，模型版本可锁定）；BGE 嵌入 + 本地向量库 + BM25 混合；Gitee OpenAPI / GitHub REST API；Maven + JDK 8/17/22 + Testcontainers；SQLite + 文件系统；Docker Compose 一键启动。

vibe coding 的工程化约束

任务书技术要求第一条列出「熟练掌握 vibe coding」。vibe coding 指不逐行审查 AI 产出的代码，而是描述意图、运行观察、依据结果迭代，将正确性判断从读代码转移到看行为。

这一模式的前提，是行为反馈足够快且可信。

而 Easy-Es 恰是该前提最脆弱的场景：改动链路漏任何一层的结果是 能编译但语义错误；全量测试需 Testcontainers 起真实 ES 耗时 5 至 15 分钟，若每次迭代都等 15 分钟，日迭代次数被压到十几次。典型失败形态是「跑通了」但结果不对。

本项目不把 vibe coding 当作工具使用，而作为需被工程化约束的对象：

vibe coding 的缺陷	技术响应
反馈慢，迭代次数被压缩	M5 分层验证(L0 / L1 / L2 分档指标见 U9 至 U11)
反馈不可信，「跑通」误判为「对了」	P1 客观信号优先，LLM 自评不作放行依据
无终止条件，易陷循环	M7：重复输出相似度检测即熔断，重试上限 15 次
责任边界模糊	P2：三处强制人工门禁，每步产出落盘可回溯

结论

本项目的实质，是为 vibe coding 补上其在 Java 生产仓库中缺失的一环：可信、快速、分层的验证反馈。

模型能力给定时，vibe coding 的产出质量上限由验证反馈质量决定。

实施计划

仓库归属与结项合规

任务书「项目成果仓库」字段指定为 `https://gitee.com/dromara/easy-es`。据此，AI 生成的功能模块 PR 必须提交至该仓库，这也是结项的核心依据；Harness 系统代码的归属任务书未作规定，需另行确认(Q4)。

成果	归属仓库	合规依据
AI 生成的功能模块 PR (结项核心依据)	<code>gitee.com/dromara/easy-es</code> 项目详情页指定	满足结项标准「以 PR/MR 形式提交到项目所在的开源组织仓库中并完成合并」
Harness 系统代码 (过程产物，计入代码贡献量)	优先争取 <code>gitee.com/dromara/easy-es-harness</code> ；退而求其次放个人仓库	退至个人仓不影响结项资格，仅影响代码贡献量评分

Harness 不并入 easy-es 主仓：该项目为 Apache-2.0 纯 Java 项目，混入数千行 Python 编排代码会污染主仓结构并干扰既有 CI。

与上游仓库的边界

源码、Issue、构建与测试信号均为只读；代码改动只在独立工作副本的 `feature` 分支上进行，不直接写主仓；PR 提交前经人工门禁。PR 评论写入为最高危操作，默认关闭自动评论，仅在人工确认后单次执行，全局频次上限为每日不超过 3 次。

系统形态

产出要求 1 明确要求「基于 Skills 的爬虫模块」，故本项目以 Skills 规范封装四个技能单元(`issue-crawler` / `capability-index` / `design-draft` / `pr-composer`)，其中 `issue-crawler` 即该条所要求的爬虫模块(对应 M1 感知层)。同时提供纯 CLI 入口，不依赖特定 Agent 运行时亦可独立运行；通过 Docker Compose 一键启动，便于复现与验证。

四个月进度安排

阶段划分

任务书「开发周期」为 4 个月，「支持架构」为 RISC-V64。据此将周期划分为四个阶段，覆盖 W1 至 W17，其中 W17 整周预留为缓冲(见「缓冲与沟通」)。RISC-V64 的验收口径任务书未作说明，列为待确认事项 Q1。

阶段	主题	覆盖周次	阶段产出
第 1 月	环境搭建与数据感知	W1 至 W4	可编译环境、基线数据报告、1 个手工 PR、爬虫与分类器
第 2 月	能力图谱与分层验证	W5 至 W8	检索服务、选题引擎、L0/L1 验证器
第 3 月	设计生成与验证闭环	W9 至 W13	L2 验证器、设计生成器、代码生成器、生成至验证闭环
第 4 月	端到端交付与 PR 产出	W14 至 W17	交付与管控模块、完整流水线、2 个 PR、文档与结项材料

周度计划

周次	主题	任务	产出
W1	环境基线	装 JDK 8/17 + Maven；解决 7.17.28 依赖冲突；跑通编译；记录基线数据；与导师确认 Q1/Q2/Q3(Q4 顺带确认)	可编译环境 + 基线数据报告
W2	破冰 PR	读源码与 PR 模板；手工提交 1 个 Easy-Es PR；跑通全量测试；精读《避坑指南》补 ES 知识	1 个 PR(D1)+ 偏好记录
W3	M1 感知层(U1)	Gitee / GitHub API 增量抓取；SQLite 落库	爬虫模块 + 原始候选库
W4	M1 分类器(U2)	规则层实现；人工标注 200 至 300 条样本；LLM 分类层调优	分类器 + 标注集 + 准确率报告
W5	M2.1 能力图谱(U3)	源码两级切分；BGE + BM25 混合索引；以 3.x 源码为唯一事实来源	检索服务 + 索引
W6	M2 选题引擎(U4)	结构化评分字段；检索命中率测试集验证(含 #143/#135/#157)	选题引擎 + 命中率报告
W7	M5-L0(U9)	编译 + 静态检查 + 依赖校验流水线	L0 验证器(小于 30s)
W8	M5-L1(U10)	定向单测执行器(-pl + -Dtest)	L1 验证器(小于 2min)
W9	M5-L2 + 门禁(U11)	Testcontainers 起真实 ES；三条测试线全量回归；客观信号采集	L2 验证器 + 信号采集器
W10	M3 设计生成(U5)	架构 + 详细设计生成；改动链路约束注入	设计生成器 + 2 份样例设计
W11	M3 Review(U6)	一致性校验(设计声明 vs 实际 diff)；AST 对称性检查	Review 校验器
W12	M4 代码生成(U7、U8)	基于 diff 定向生成；单元测试生成(社区 95% 覆盖率硬约束)	代码生成器
W13	M4 + M5 联调	生成至验证闭环跑通；迭代策略调优	闭环可运行 + 迭代统计
W14	M6 + M7(U12、U13)	分支 / Commit / PR 自动化；五项熔断实现	交付模块 + 管控层
W15	端到端串联	全链路打通；三处人工门禁接入(选题 / 设计 / PR 提交前)；状态机持久化与回放	完整流水线 + 端到端演示
W16	实战产出	运行流水线，产出第 1 至 2 个 AI 挖掘的功能 PR (需求可溯源至 Issue 编号)，提交审核	2 个 PR(核心交付)
W17	文档与结项	使用说明文档；演示脚本；结项报告	文档 + 结项材料

若中选晚于 2026 年 10 月，按 N6 预案执行：W16 的 PR 产出前置至 W14 末启动，W3 至 W6 压缩至 3 周。

缓冲与沟通

每周预留约 4.5 小时(15%)应对 Debug 与返工；W2 末、W9 末、W13 末设检查点；W17 整周为整周缓冲。

降级预案：若 W13 闭环未跑通，砍掉 U6 的 AST 检查，仅保留文件级一致性校验；若 W16 仅产出 1 个 PR，优先保证质量而非数量。

沟通：每周向导师同步进度(文字简报 + 关键产出链接)；W2 / W9 / W13 / W16 请求评审与纠偏；技术卡点超过 3 天立即求助。

需导师确认事项

#	问题	默认假设与方案
Q1	「支持架构 RISC-V64」的验收口径	「开发周期」与「支持架构」同为项目详情页系统字段，且 2026 年活动以 RISC-V 为生态重心，推测其指 Harness 系统本身的可移植性；官方 README 已宣称支持 RISC-V，但 Elastic 无 riscv64 构建。 方案：① Harness 全栈架构无关(Python + Docker)，在 x86_64 / aarch64 验证；② 测试目标设计为可插拔，RISC-V64 实例就绪即可挂载；③ 若需端到端验证，申请云实例并列为冲刺档
Q2	产出 5 中「高质量 PR」的判定是「生成并提交」还是「被合并」？若需合并，哪些模块属禁区	假设需合并；索引托管与数据迁移路径视为禁区
Q3	开发基线(master 分支还是 v3.0.2 tag) 与 PR 目标仓库(Gitee 还是 GitHub)	假设 Gitee + 与导师确认的基线。另需确认：GitHub 镜像有 104 个 open issue，M1 是否双源抓取
Q4	Harness 代码能否在 Dromara 组织下新建 easy-es-harness 仓库	优先争取组织下新仓；若不支持退至个人仓库，不影响结项

附：Issue #171(2026-08-11)指出 tomcat-embed-core:9.0.82 存在经 re-bundling 隐藏的 CVE，至今无维护者回复，与官网「墨菲安全扫描零风险」表述有出入。该条目涉及依赖升级，属敏感路径，不作为默认选题，仅在导师明确认可后纳入。

11 · DELIVERABLES

预期产出

#	交付物	对应产出要求
D1	Easy-Es 功能 / 修复 PR × 1(W2, 手工, 用于建立基线与对齐偏好)	前置准备
D2	AI 自动生成的功能模块 PR × 2(W16, 通过全量测试, 需求可溯源至 Issue 编号)	产出 4、5
D3	Harness 系统源码(M1 至 M7, 含 U1 至 U13 实现单元)，Docker Compose 一键启动	产出 1、2、3、4
D4	Skills 技能单元 × 4(issue-crawler / capability-index / design-draft / pr-composer)	产出 1
D5	使用说明文档 + 演示脚本	产出 5
D6	能力图谱构建产物 + 检索命中率测试报告	产出 2
D7	分层验证基线数据报告(L0/L1/L2 耗时与拦截率实测)	产出 4
D8	结项报告	组委会要求

验收标准

验收口径以任务书为准。产出要求 5 要求「基于真实需求演示生成至少 1 个 高质量 PR」；项目目标第三条要求「至少实现 1 至 2 个由 AI 自动挖掘并生成的、高质量的功能模块 PR」。据此分为保底、达标、冲刺三档。

分档标准

档位	标准
保底 (必须达成)	同 §07 基础实现目标
达标 (目标)	保底 + D2 的 2 个 PR 均通过测试并提交，其中至少 1 个被合并；M5 达成设计指标(L0 小于 30s / L1 小于 2min / 单 PR 小于 40min)
冲刺	达标 + 检索命中率 $\geq 80\%$ ；单 PR 平均迭代次数 ≤ 10 ；完成 RISC-V64 环境兼容性验证(视 Q1 结论)

量化指标

其中「AI 生成 PR 的需求可溯源率 100%」直接对应产出要求 5 的「基于真实需求」：可溯源即排除构造演示需求走通流水线的做法，是任务书对该条的硬性限定。

指标	目标值
候选需求池分类器准确率	$\geq 85\%$
能力图谱检索命中率(30 至 50 条测试集，含 #143 / #135 / #157)	$\geq 80\%$
L0 拦截率	$\geq 60\%$
L0 + L1 合计拦截率	$\geq 85\%$
单 PR 平均迭代次数	8 至 15 次
单 PR 端到端总耗时	小于 40 分钟
AI 生成 PR 通过全量测试的比例	$\geq 50\%$
AI 生成 PR 的需求可溯源率	100%

范围边界与可行性结论

基础阶段明确不包含的内容

为避免范围蔓延，以下内容明确不在基础实现目标内：

不包含	原因
通用化框架	P5 原则，仅针对 Easy-Es 单仓库；不做「以后也能用于其他项目」的抽象
索引托管与数据迁移路径的改动	属高风险敏感路径，设为禁区 (Q2)
移除人工门禁 / 转向全自治	与任务书「质量把关」条要求的三阶段人工审核冲突 (\$04)
Web UI	以 CLI + 结构化产物为主，降低范围与维护成本
多模型自动路由与成本优化	与本项目目标无关，属过度工程
依赖升级类改动 (如 Issue #171)	涉及依赖版本，需导师明确认可后才纳入

可行性结论

技术路径可行。半自动而非全自治的依据来自任务书：其「质量把关」条要求设计、编码、测试三阶段的产出均经「自动 Review + 人工审核」双重把关 (\$04)。技术上的必要性在于：Easy-Es 漏改的表现是「能编译但语义错误」(\$03)，通过测试不等于语义正确，因此决策点必须留人。核心矛盾有解 (\$08)，分层验证把错误发现成本从 15 分钟压到 30 秒，使 6 至 35 次迭代 (\$06 N3) 在 40 分钟内完成；主要难点逐条有对策 (\$06)，已有基础可直接复用 (\$06)。

主要不确定性为导师侧的三项确认 (Q1 架构口径、Q2 PR 判定标准、Q3 基线)，均已在 §10 提出，不影响主体方案，可在 W1 内收敛。